

1571 Multi-Speeder V3

Manual

1571 Switchless 8x Multi-Floppy-Speeder V3 (512 KB) for the Commodore C128DCR with DolphinDos26 for the 1571, the C128 kernal DolphinDos26 and the MS-Tools with build instructions and bill of materials

Frank Eggen – as of 11 November 2026

Contents

1 Introduction....3
1.1 What is the 1571 Multi-Speeder V3?....3
1.2 About this manual....3
1.3 Files....4
2 The hardware....5
2.1 Overview....5
2.2 The Multi-Speeder board....5
2.3 Memory map of the 1571....7
2.4 The parallel port adapter....8
2.5 The kernal switcher....10
2.6 PIA types....12
3 Switching the banks....13
4 DolphinDos26 for the 1571....14
4.1 Disks in the drive RAM: single-sided and double-sided....14
4.2 Clock: 2 MHz and 1 MHz....15
5 The C128 kernal DolphinDos26....16
6 Testing and working with the MS-Tools....17
7 Building it yourself....18
7.1 What you need....18
7.2 Having the boards made....18
7.3 Assembly....18
7.4 Programming the GAL....19
7.5 Programming the microcontroller....20
7.6 Preparing the EPROMs....21
7.7 Installation in the C128DCR....23
7.8 First start-up....24
8 Bill of materials (BOM)....26
8.1 Multi-Speeder board....26
8.2 Parallel port adapter....27
8.3 Kernal switcher....27
8.4 Cables and installation....27
9 Appendix....28
9.1 RAM banks in your own programs....28
9.2 Troubleshooting....28
9.3 Origin and acknowledgements....28
9.4 Disclaimer....29
9.5 Supporting the project with a donation....30
9.6 Copyright and terms of use....30

1 Introduction

1.1 What is the 1571 Multi-Speeder V3?

The **1571 Switchless 8x Multi-Floppy-Speeder V3** speeds up the built-in 1571 drive of the **Commodore C128DCR**. It consists of three boards that all disappear inside the case: the Multi-Speeder board itself in the CPU socket of the 1571, a parallel port adapter in the socket of CIA 2 and a kernal switcher in the kernal socket of the C128. The user port stays free.

- **8 DOS banks** of 64 KB for the 1571 in a 512 KB flash EPROM 29F040, switchable by DOS command without a switch (“switchless”)
- **8 kernal banks** for C128 and C64 mode in the kernal switcher; they are always switched together with the DOS bank
- **512 KB of RAM** in the drive, mapped as 64 banks of 8 KB at $6000–$7FFF
- a **PIA 6821/W65C21** as an 8-bit **parallel port** to the computer and for selecting the RAM banks
- RAMBoard-compatible for nibbler programs

With **DolphinDos26** for the 1571 and the **C128 kernal DolphinDos26**, the C128DCR offers the same functions as a C64 with 1541 and Multi-Speeder: parallel transfer, track cache, whole disks in the drive RAM, a RAM disk, the MS-Addon-API and the MS-Tools – plus double-sided disks.

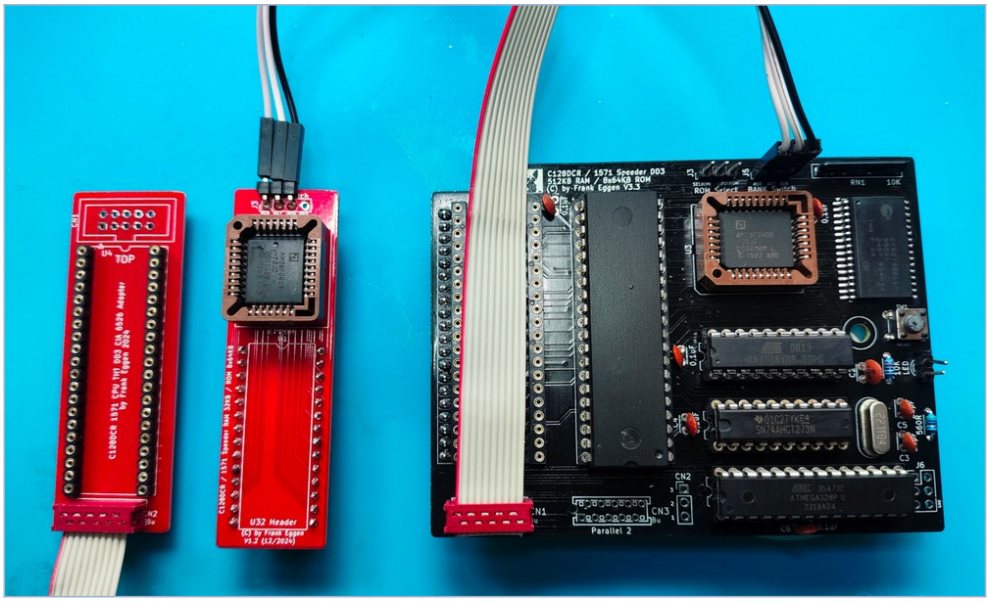


Figure 1: The three boards of the 1571 Multi-Speeder V3: kernal switcher, parallel port adapter and Multi-Speeder board

1.2 About this manual

This manual describes the 1571 Multi-Speeder V3 in the C128DCR: the hardware, switching the banks, the DolphinDos26 firmware for the 1571, the C128 kernal, the MS-Tools and **building it yourself** with a bill of materials. Operating DolphinDOS and the commands of DolphinDos26 are described in detail in the **DolphinDos26 manual**; this manual covers what is special about the 1571.

Notation: input is shown in typewriter font, e.g. `@I:4@RNROM`. As with the commands, the banks are counted from 1 to 8; bank n starts at address (n − 1) × $10000 in the EPROM.

1.3 Files

The firmware belongs to the DolphinDOS26 project (github.com/FraEgg/DolphinDOS26). Everything needed to build it – Gerber files, schematics, bill of materials, GAL and ATmega files – is in the 1571 Multi-Speeder project together with this manual and the ROM files (github.com/FraEgg/Switchless-Multi-Speeder-Commodore-C128DCR-1571).

File	Content	Checksum (MD5)
`C1571_DOLPHINDOS26.bin`	64 KB: DolphinDos26 for the 1571, bank 4 of the 1571 EPROM	`947041a25c15acaa979fa87181f07ab1`
`C128DolphinDos26-US.BIN`	16 KB: C128 kernal DolphinDos26 (C128 mode), international and USA	`859493314e98777a595d0f2a039e6800`
`C128DolphinDos26-DE.BIN`	16 KB: C128 kernal DolphinDos26 (C128 mode) with the German keyboard driver (DIN)	`20aea19be1369bceddc0d6dccb7d373f`
`C64IIDolphinDos26.bin`	16 KB: C64 BASIC and C64 kernal DolphinDos26 for the C64 mode of the C128	`54fa1120daf6baafe39c31495a22f448`
`C1571_C128DCR_29F040.BIN`	512 KB: template for the EPROM U3 (29F040): Commodore DOS 318047-01 in banks 1, 2 and 5–8, original DolphinDOS 3 in bank 3, DolphinDos26 in bank 4	`bba2f64edaf2e15af82057bc07ab5014`
`C128DCR_KERNAL_29F040-DE.BIN`	512 KB: template for the EPROM of the kernal switcher U32 (29F040), German version, banks matching `C1571_C128DCR_29F040.BIN`	`e467abc52085c8e46d4ff0734ac0e21c`
`C128DCR_KERNAL_29F040-US.BIN`	512 KB: template for the EPROM of the kernal switcher U32 (29F040), international and US version	`8068cd0346f8fc2a34d5ddabe582a561`
`ms-tools.d64`	MS-Tools disk (folder `MS-Tools`)	`74729c247452ed1b059fa174bf23677c`
`ATF16V8_PAL_Adressendecoder_V3_0PIA_1571.jed`	Address decoder for the GAL U7	`f09d794c1f367be02726e5ab61ae2fba`
`CBM_drive_romswitch.328p_20MHz.hex`	Firmware of the microcontroller U6 (also runs at 22.1184 MHz)	`ca6efadd473cf45f885064e1dcf4018c`

> **Note:** Besides DolphinDos26, the template `C1571_C128DCR_29F040.BIN` contains the original Commodore DOS 1571CR and the original DolphinDOS 3; it can be programmed directly. To match it, the templates `C128DCR_KERNAL_29F040-DE.BIN` (German version) and `C128DCR_KERNAL_29F040-US.BIN` (international and USA) for the kernal switcher contain the Commodore kernals, the original Dolphin kernals and the DolphinDos26 kernals; they can be programmed directly as well (chapter 7.6). Your own images can be assembled with the browser tool MS-ROM-Builder-C128DCR (chapter 7.6).

2 The hardware

2.1 Overview

Board	Sits in the C128DCR	Task
Multi-Speeder board	in the socket of the 1571 CPU (U101); the 6502 sits on top of the board	DOS EPROM with 8 banks, 512 KB RAM, PIA, microcontroller, address decoder
Parallel port adapter	in the socket of CIA 2 6526 (U4); the CIA sits on top of the adapter	takes port B of the CIA (as on the user port) to the Multi-Speeder over a 10-way cable
Kernal switcher	in the socket of the kernal ROM (U32)	EPROM with 8 banks for C128 and C64 mode, switched via A16–A18 by the Multi-Speeder

> **Important:** With the Multi-Speeder board, the original DOS ROM of the 1571 (**U102**) must be **removed without replacement**. The Multi-Speeder supplies the DOS from its own EPROM; a remaining U102 causes an address conflict and the drive does not work.

2.2 The Multi-Speeder board

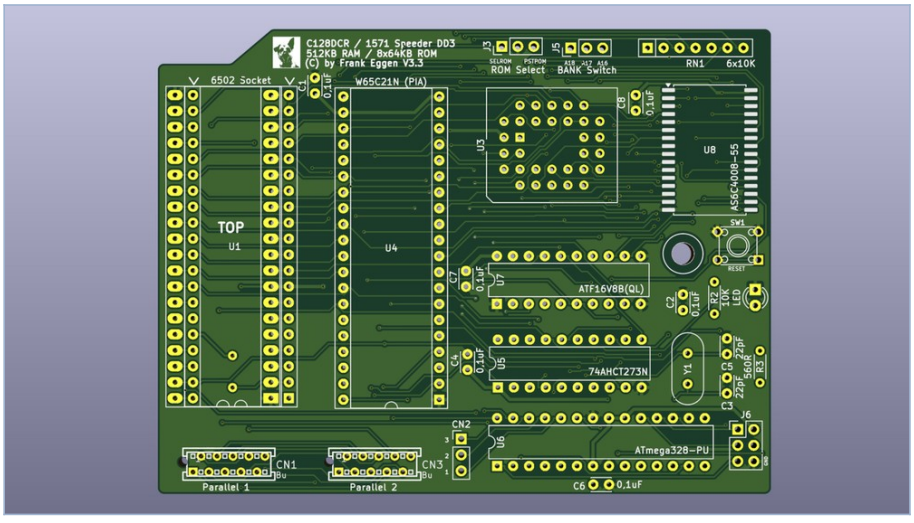


Figure 2: Multi-Speeder board V3.3 (component side)

Component	Task
U1, U2	U1: 40-pin socket for the drive's 6502; U2: 40-pin DIP plug with which the board sits in the CPU socket U101
U3	Flash EPROM 29F040 (PLCC32, 512 KB) with 8 DOS banks of 64 KB; alternatively 27C040 or SST39SF040
U4	PIA W65C21N or 6821: port A is the parallel port, port B (PB0–PB5) selects the RAM bank
U5	74AHCT273: latches the byte from the data bus for the microcontroller
U6	Microcontroller ATmega328P with crystal Y1 (22.1184 MHz): recognises `n@RNROM` and switches A16–A18

Component	Task
U7	GAL ATF16V8B: address decoder for EPROM, RAM and PIA
U8	512 KB SRAM AS6C4008 (SOP32)
RN1	Pull-up resistors 6 × 10 kΩ for the RAM addresses A13–A18 (see appendix “RAM banks in your own programs”)
D1, R3	LED: blinks the number of the new bank when switching
SW1, R2, C2	Reset button for the microcontroller

Connector	Pinout and purpose
CN1, CN3	Parallel port (Micro-MaTch, 10-way): PA0–PA7, CA1, CA2; two equivalent positions for routing the cable
J5 BANK Switch	A16, A17, A18 to the kernal switcher (J3 there); this switches the kernal with the DOS bank
J3 ROM Select	Pin 1 SELROM, pin 2 GND, pin 3 RSTROM: briefly connecting pin 3 to GND resets to bank 1, pin 1 to GND advances one bank
J6 SPI	ISP connector for programming the ATmega on the board
J2	Solder jumper for the EPROM type: pad 2 (middle) is A18; 29F040 and SST39SF040: pad 1–2, 27C040: pad 2–3 (see chapter 7.6)
J4 RESET TYPE	Solder jumper: closed, the microcontroller resets the drive when switching (default); open, no reset
J1	Solder jumper, to be opened only for a WDC 65C02 – DolphinDos26 needs the original NMOS 6502, J1 stays closed
CN2	free pins of the microcontroller (PC1–PC3), not used

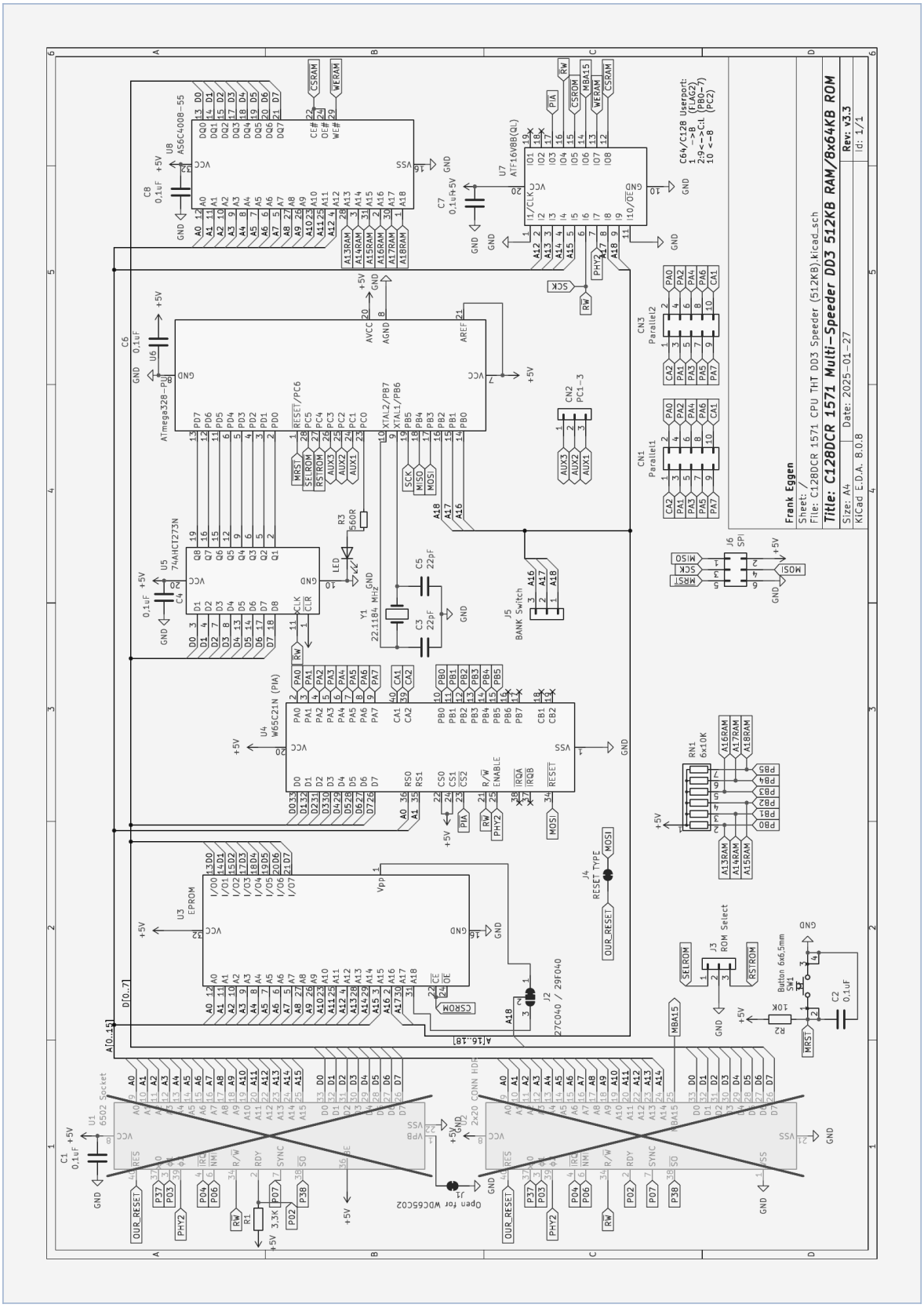


Figure 3: Schematic of the Multi-Speeder board V3.3 (turn the page)

2.3 Memory map of the 1571

Address	Area
$0000–$07FF	2 KB RAM of the 1571

Address	Area
$1800–$180F	VIA 1 6522 (serial bus)
$1C00–$1C0F	VIA 2 6522 (drive control)
$2000–$2FFF	Floppy controller WD1770 (MFM)
$3000–$3FFF	**additional DOS ROM** from the active bank (MS-Addon of DolphinDos26)
$4000–$4FFF	CIA 6526 of the 1571
$5000–$5003	**PIA** W65C21/6821
$6000–$7FFF	**8 KB RAM window**: one of 64 banks of the 512 KB SRAM, logical bank 0 after power-on
$8000–$FFFF	**DOS ROM** from the active bank

In the EPROM each bank occupies 64 KB; the ranges bank start + $3000–$3FFF and + $8000–$FFFF are mapped, i.e. at the same addresses as in the CPU. A 32 KB DOS ROM therefore belongs in the upper half of the bank.

2.4 The parallel port adapter

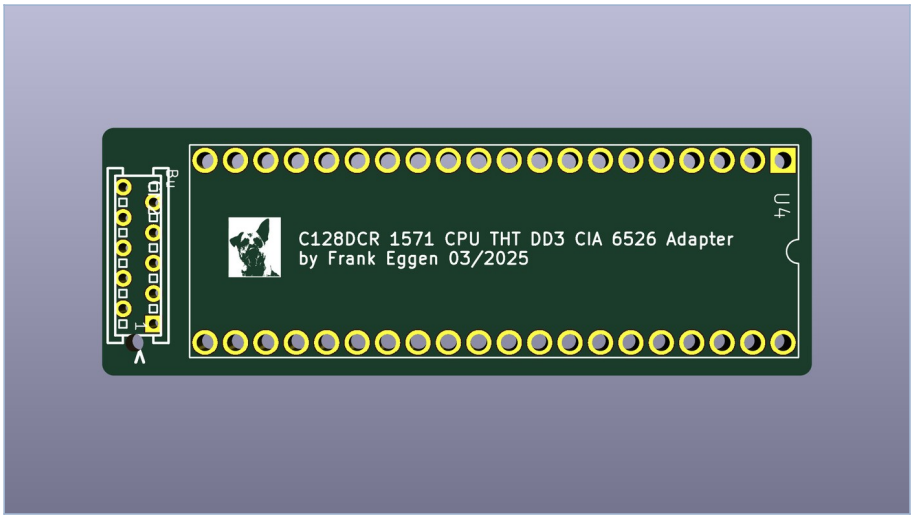


Figure 4: Parallel port adapter V3.3 for socket U4 (CIA 2)

The adapter is fitted in place of CIA 2 (U4), and the CIA goes into the adapter's socket. It taps port B and the handshake lines – the same signals that are on the user port of the C64 and C128 – and takes them over a 10-way Micro-MaTch cable to CN1 or CN3 of the Multi-Speeder board. No external parallel cable is needed. The adapter is only required for the parallel transfer of DolphinDOS.

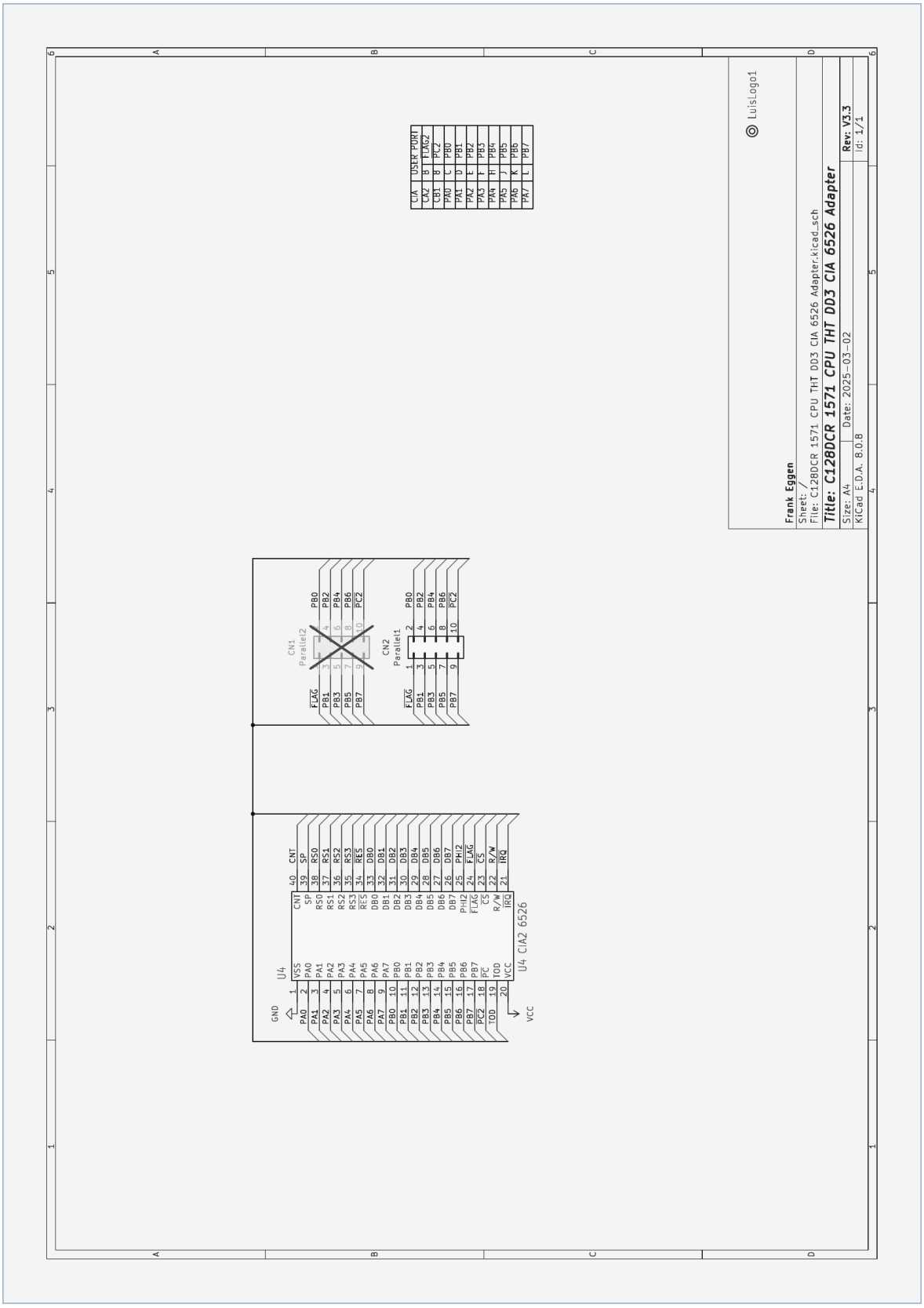


Figure 5: Schematic of the parallel port adapter V3.3 (turn the page)

2.5 The kernal switcher

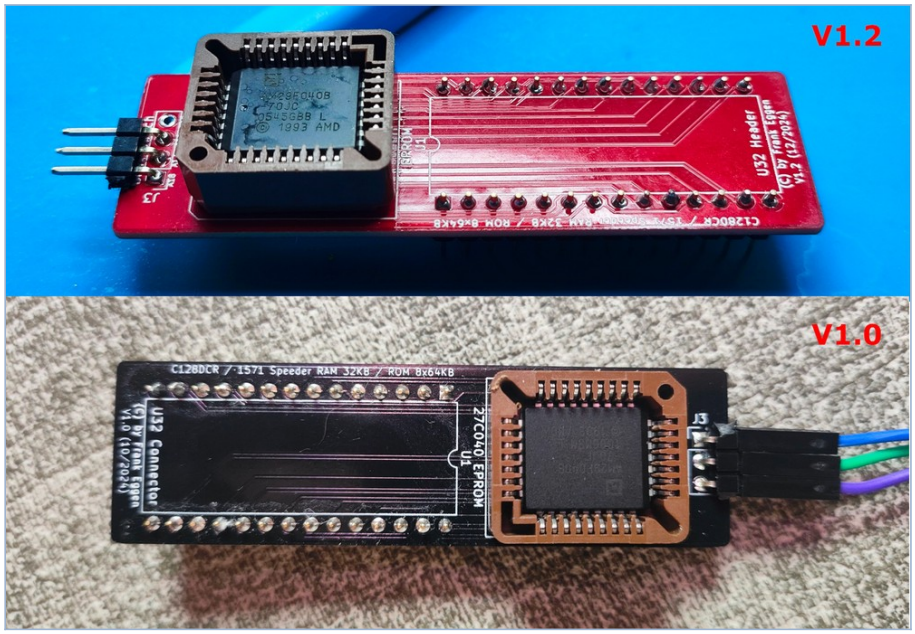


Figure 6: Kernal switcher for socket U32: version 1.2 (top) and 1.0 (bottom)

The kernal switcher carries a second 512 KB EPROM with 8 banks for the C128 and replaces the kernal ROM in U32. Its connector J3 is connected to J5 of the Multi-Speeder board (A16, A17, A18). The same bank is therefore always active as in the drive: bank 4 in the drive means bank 4 in the kernal switcher. The fourth connection (A15) of version 1.2 stays free in the C128DCR.

Only the upper half of each bank of the EPROM is used, as in the original 32 KB ROM U32:

EPROM address within the bank	Content	File with DolphinDos26
$0000–$7FFF	not used	–
$8000–$9FFF	C64 BASIC V2 (C64 mode)	`C64IIDolphinDos26.bin` (16 KB, BASIC and kernal)
$A000–$BFFF	C64 kernal (C64 mode)	(second half of `C64IIDolphinDos26.bin`)
$C000–$FFFF	C128 kernal and system routines	`C128DolphinDos26-US.BIN` or `C128DolphinDos26-DE.BIN` (16 KB)

> **Note:** When switching, only the drive is reset, not the C128. After changing the bank, restart the C128 with the reset button so that the matching kernal becomes active.

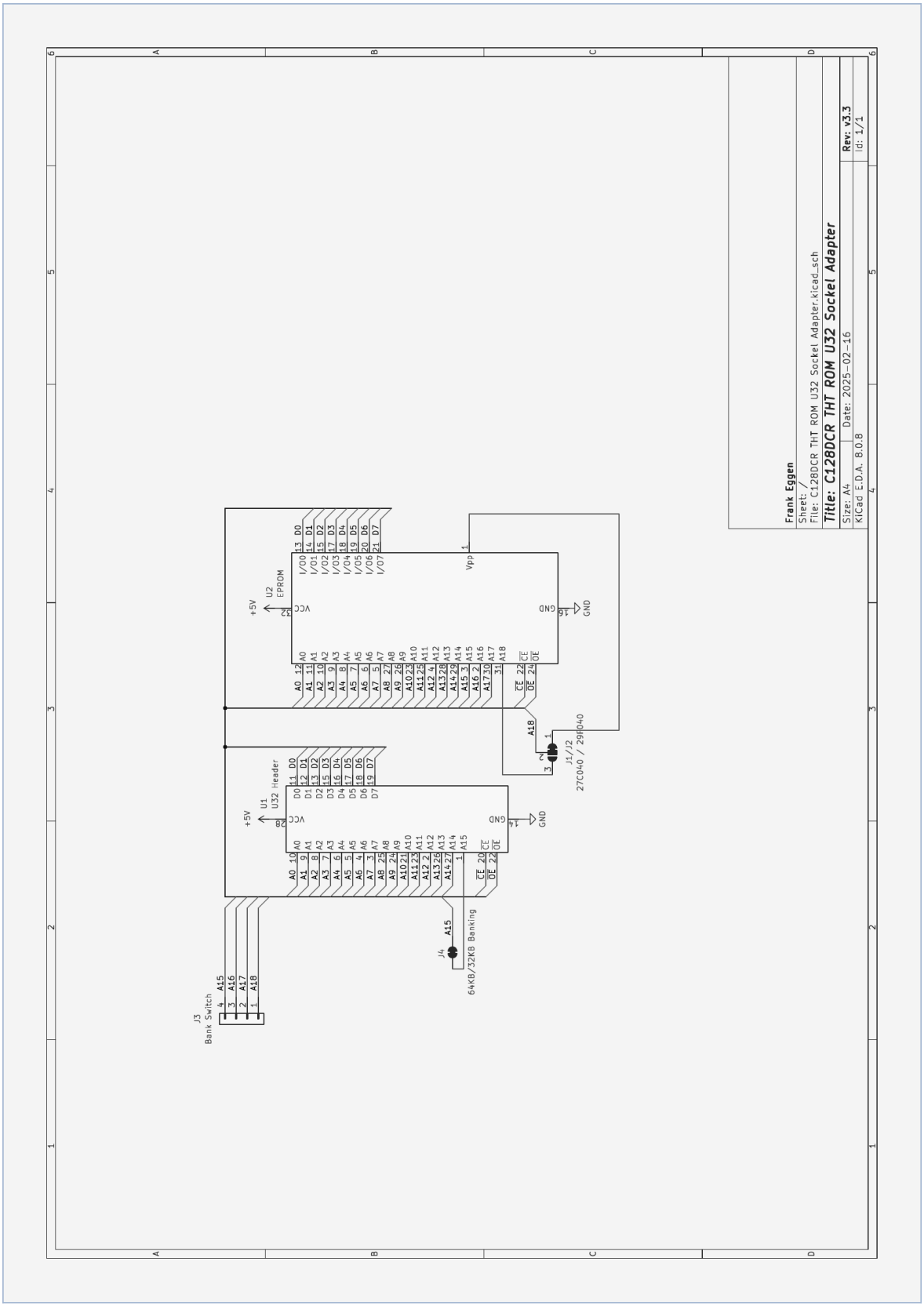


Figure 7: Schematic of the kernal switcher V3.3 (turn the page)

2.6 PIA types

These PIA chips have been tried: **W65C21N6TPG-14** or **W65C21S6TPG-14** (Western Design Center), **MC68B21P** (Motorola, 2 MHz), **EF68B21** (STMicroelectronics, 2 MHz) and **R65C21P2** (Rockwell, 2 MHz). With DolphinDOS the 1571 runs at 2 MHz; the PIA must handle this clock.

3 Switching the banks

The microcontroller watches the data bus of the 6502. When it recognises the string `n@RNROM` (n = 1–8), it switches the DOS and kernal bank to n, resets the drive and blinks the LED n times. It stores the selected bank in its EEPROM; it also applies after power-off.

```
@I:4@RNROM
OPEN 1,8,15,"I:4@RNROM":CLOSE 1
LOAD"4@RNROM",8,1
```

Bank	Command	EPROM range	Drive (example)	Kernal switcher
1	`1@RNROM`	$00000–$0FFFF	CBM DOS V3.1 1571CR (318047-01)	Commodore kernals
2	`2@RNROM`	$10000–$1FFFF	CBM DOS V3.0 1571	Commodore kernals
3	`3@RNROM`	$20000–$2FFFF	DolphinDOS 3 1571 (original)	Dolphin kernals (original)
4	`4@RNROM`	$30000–$3FFFF	**C1571DolphinDos26**	**C128DolphinDos26** and **C64IIDolphinDos26**
5–8	`5@RNROM` … `8@RNROM`	$40000–$7FFFF	free for further DOS versions	matching kernals

The first form works with a DolphinDOS kernal, the other two with any kernal. `0@RNROM` disables switching until the next power-on. This is useful if a program happens to transfer this string.

> **If switching does not work:** At 2 MHz the microcontroller can miss individual commands. Then simply repeat the command. It works reliably with the **22.1184 MHz** crystal (instead of 20 MHz) and a **74AHCT273** as U5. If the drive hangs completely, a jumper on J3 from pin 3 to pin 2 (GND) resets to bank 1.

4 DolphinDos26 for the 1571

C1571DolphinDos26 is DolphinDos26 for the built-in 1571 of the C128DCR (1571CR). It is based on the original DolphinDOS 3 for the 1571, plus the MS-Addon of DolphinDos26 with the same commands and the same API as on the 1541. After `@I:4@RNROM`, `@` reports the firmware as `73,DOLPHINDOS26 1571,00,00`.

As on the 1541, you get: parallel transfer, track cache, the X commands, `CDR` and `CDW` (read a disk into RAM and write it back), the RAM disk `CDM`, fast loading and saving from the RAM disk and the MS-Addon-API. Details of these commands are in the DolphinDos26 manual.

4.1 Disks in the drive RAM: single-sided and double-sided

The **1541** – and therefore the C64 – writes a disk on **one side** only: 35 tracks (with DolphinDOS also 40), 664 free blocks, as an image a D64 file. In the 1571 mode of the C128, the **1571** can use **both sides**: 70 tracks, 1328 free blocks, as an image a D71 file. A 1541 cannot fully use a double-sided disk.

Most disks are written single-sided so that they stay compatible with the 1541 – many programs for the C128 are also supplied that way. In C64 mode the 1571 works like a 1541 with one side anyway.

The 512 KB RAM of the Multi-Speeder offers two **RAM pages**: each holds one single-sided disk. A double-sided disk occupies both RAM pages.

4.1.1 Single-sided disks (as on the 1541)

Command	Effect
`@CDR1` / `@CDR2`	read the disk into RAM page 1 or 2 (35 tracks; in 1541 mode also 40)
`@CDW1` / `@CDW2`	write RAM page 1 or 2 to the inserted, formatted disk – as many tracks as CDR read
`@CDM1` / `@CDM2`	switch on the RAM disk: drive 8 works with the content of RAM page 1 or 2
`@CDM0`	RAM disk off, back to the disk

A typical sequence: insert the source disk and read it into RAM page 1, insert the target disk and write it (can be repeated as often as you like), then work with the RAM disk from page 1 and return to the disk:

```
@CDR1
@CDW1
@CDM1
@CDM0
```

The two RAM pages are independent: page 1 and page 2 can hold two different disks. The content is kept until the drive is switched off or the page is written again.

4.1.2 Double-sided disks (1571)

C1571DolphinDos26 puts a double-sided disk into both RAM pages: side 0 of the disk (tracks 1–35) into RAM page 1, side 1 of the disk (tracks 36–70) into RAM page 2.

Command	Effect
`@CDRD`	read a double-sided disk into both RAM pages
`@CDWD`	write both RAM pages to a double-sided disk
`@CDMD`	switch on the RAM disk from both RAM pages (tracks 1–70)
`@CDM0`	RAM disk off, back to the disk

For these commands the firmware switches DolphinDOS into 1571 mode and back afterwards; they therefore also work in C64 mode. The target disk for `CDWD` must be formatted double-sided (`@N:NAME,ID` in C128 mode). `@CDRD` overwrites both RAM pages, including what was previously read with `@CDR1` or `@CDR2`. In the MS-Addon-API, `E0 OPEN` and `E6 SECTOR` also accept the value 3 as side for a double-sided disk.

> **Difference from the 1541:** On the 1571, DolphinDOS accepts tracks 36–40 only in 1541 mode and only on disks it has formatted with 40 tracks itself. On track 36 of a 35-track disk the firmware reports `21,READ ERROR`.

4.2 Clock: 2 MHz and 1 MHz

C1571DolphinDos26 behaves like the original DolphinDOS 3: **the drive always runs at 2 MHz**, also in the C64 mode of the C128. DolphinDOS needs the 2 MHz for the track cache; the parallel transfer uses handshaking and does not depend on the clock. Programs that load their own time-critical code into the drive (fast loaders, copy programs) run twice as fast at 2 MHz and may then fail. These commands are available for that:

Command	Clock	Effect
`@X-`	1 MHz	DolphinDOS off: no track cache, no parallel transfer; the drive behaves like a 1571 with Commodore ROM in 1541 mode
`@X+`	2 MHz	DolphinDOS with all functions again
`@I:1@RNROM`	1 MHz	Commodore ROM for programs that expect an unmodified drive (back with `@I:4@RNROM`)

Measured in the VICE emulator: loading 30 KB with DolphinDOS parallel takes about 5.3 s, after `@X-` serially at 1 MHz about 106 s. The firmware does not depend on the TV standard; parallel transfer and JiffyDOS work with PAL and NTSC, in C128 and C64 mode.

5 The C128 kernal DolphinDos26

C128DolphinDos26 comes in two versions that differ only in the keyboard:

File	Market	Keyboard
`C128DolphinDos26-US.BIN`	international, USA	US screen editor (318020-05), CAPS LOCK as usual
`C128DolphinDos26-DE.BIN`	German-speaking countries	additionally the German keyboard driver (DIN) from the kernal 315078-03: the CAPS LOCK key is the ASCII/DIN key – locked down for the DIN keyboard with the German umlauts and the sharp s, swapped Y/Z and dead keys, up for ASCII as in the US version. To show the umlauts on screen, the C128 needs the German character ROM 315079-01.

C128DolphinDos26 brings the transfer methods of DolphinDos26 to C128 mode. On every access it picks the fastest method the device supports:

- DolphinDOS **parallel** to DolphinDos26 drives (1571 and 1541 with Multi-Speeder), shown as `LOADING DD:`
- **JiffyDOS** to JiffyDOS drives and SD2IEC, with block-wise fast loading into any memory bank (`BLOAD`, `BOOT`), shown as `LOADING JF:`
- burst drives (1571, 1581) keep the Commodore fast mode, all other devices run serially
- @ commands as with DolphinDOS (`@`, `@befehl`, `@$`, `@8` to `@11`) and function keys as with the C64 kernal, including the SD2IEC keys on the C= level
- the Commodore fixes of 318020-05 that the original Dolphin C128 kernal lacks, e.g. `BOOT"datei"`



Figure 8: Start-up screen of the C128 with the kernal C128DolphinDos26 (40-column screen)

Below the start-up message are two DolphinDos26 lines and `PAL SYSTEM` or `NTSC SYSTEM`. Tape and kernal RS-232 have been removed: OPEN to device 1 or 2 reports `ILLEGAL DEVICE`. For C64 mode, `C64IIDolphinDos26.bin` goes into the same bank of the kernal switcher; in C64 mode it offers DolphinDOS parallel, JiffyDOS and the SD2IEC function keys.

6 Testing and working with the MS-Tools

The **MS-Tools** on the disk `ms-tools.d64` are used to test the Multi-Speeder; they replace the former diagnostic program. They run in the **C64 mode** of the C128 (`GO 64`) with the 1571 as with the 1541.

Program	Purpose with the 1571 Multi-Speeder
`MS-STATUS`	shows the error channel of the drive; after `@I:4@RNROM`, `73,DOLPHINDOS26 1571` appears
`MS-MEMTEST`	tests all 64 RAM banks (512 KB) with several patterns – the most important test after building
`MS-FASTCOPY`	disk copy via the 512 KB RAM or a REU; detects double-sided disks by itself and copies them with 70 tracks, D71 import and export (MS-RAM only, Multi-Speeder manual, chapter MS-FASTCOPY)
`MS-RS232`	RS-232 driver for C64 mode

```
GO 64
LOAD"MS-MEMTEST",8
RUN
```

> **MS-NIBCOPY** detects the 1571 and loads its own version of its drive code (1 MHz for everything at the pace of the disk, 2 MHz for the transfer). It uses the index hole sensor of the 1571, so that every track of the copy has the same position relative to the index hole as on the original, and computes at 2 MHz in C64 mode of the C128 (Multi-Speeder manual, chapter MS-NIBCOPY). **MS-15SECCOPY** itself sets a 1571 with DolphinDOS to 1 MHz with `X-` for BACKUP and VERIFY and back to 2 MHz with `X+` afterwards.

7 Building it yourself

This chapter describes how to build the 1571 Multi-Speeder V3. The parts are through-hole, only the SRAM U8 is an SMD chip (SOP32, 1.27 mm pitch). If you have never soldered before, get help.

> **Caution:** During installation, components on the C128DCR mainboard are desoldered and replaced with sockets. This is at your own risk; unplug the mains cable first and take precautions against electrostatic discharge.

7.1 What you need

- the three boards (Gerber files version 3.3) and the parts from the bill of materials in chapter 8
- soldering station with a fine tip, desoldering braid or desoldering pump (better a desoldering station), flux
- a universal programmer for EPROM/flash (PLCC32) and GAL, e.g. XGecu T48, with PLCC32 adapter
- for the ATmega the programmer or an ISP programmer (connector J6) or an Arduino as ISP
- the firmware files from chapter 1.3 and your own ROM images for the other banks

7.2 Having the boards made

The Gerber files are in the 1571 Multi-Speeder project in the folder `hardware/gerber`:

File	Board
`v3.3_C128DCR_1571_CPU_THT_DD3_Speeder (512KB).kicad_pcb.zip`	Multi-Speeder board
`v3.3_C128DCR_1571_CIA6526_Parallel_Adapter.kicad_pcb.zip`	Parallel port adapter
`v3.3_C128DCR_THT_ROM_U32_Sockel_Kernal_Adapter.kicad_pcb.zip`	Kernal switcher

Any common PCB manufacturer can make the boards from these files (two layers, standard settings).

> **Ordering the boards:** The boards can also be ordered directly via my project at PCBWay: Switchless 8x Multi-Floppy-Speeder for the C128DCR at PCBWay

7.3 Assembly

1. First solder the **SRAM U8** (SOP32): tack one corner pin, check the alignment, then solder the remaining pins with flux and check for bridges.
2. Then the low parts: resistors R2, R3, the resistor network RN1 (dot = pin 1 to the marking), the capacitors and the crystal Y1.
3. Solder in the sockets for U3 (PLCC32), U4 (DIP-40), U5 and U7 (DIP-20) and U6 (DIP-28), notches as on the silkscreen.

4. LED D1 (long leg = anode), button SW1, the pin headers J3, J5, J6 and the Micro-MaTch connectors CN1 and CN3.
5. Set the solder jumpers: **J2** according to the EPROM type (29F040: pad 1–2, see chapter 7.6), **J4** closed (reset when switching), **J1** closed (NMOS 6502).
6. Finally the socket **U1** for the 6502 and the DIP plug **U2** on the underside, with which the board sits in the CPU socket. Best solder the plug while it is aligned in a socket so that it stays straight.
7. Parallel port adapter: the 40-pin plug for socket U4, the socket for the CIA and the Micro-MaTch connector CN2. Kernal switcher: the pin headers for U32, the PLCC32 socket, the pin header J3 and the solder jumper J2 as on the Multi-Speeder board (29F040: pad 1–2).
8. Check all solder joints with a magnifier. Before fitting the board for the first time, check with a meter that +5 V and GND are not shorted.

7.4 Programming the GAL

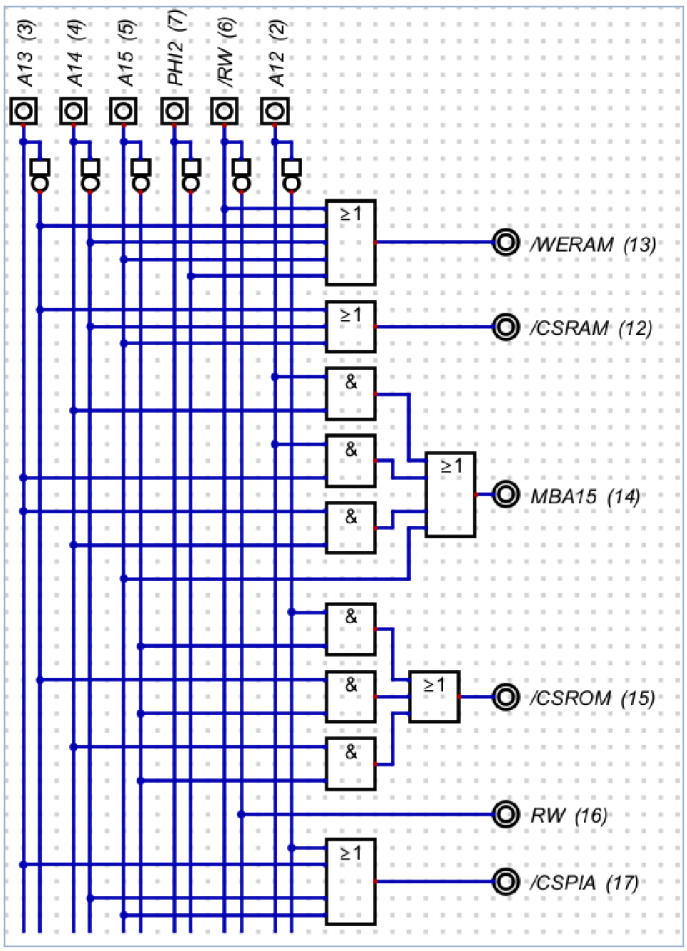


Figure 9: Logic of the address decoder in the ATF16V8B

The GAL **ATF16V8B** (U7) is programmed with the file `ATF16V8_PAL_Adressendecoder_V3_0PIA_1571.jed` (folder `hardware/gal`), e.g. with the XGecu T48 (type ATF16V8B, load file, program, verify). The decoder generates the select signals for EPROM (/CSROM), RAM (/CSRAM, /WERAM) and PIA (/PIA) as well as MBA15, which disables the ROM of the mainboard.

7.5 Programming the microcontroller

The **ATmega328P** (U6) gets the file `CBM_drive_romswitch.328p_20MHz.hex` (folder `hardware/atmega328p`). It runs unchanged with the 22.1184 MHz crystal. It is programmed either in the programmer (DIP28) or on the board via J6 (ISP). The fuse bytes are important:

Byte	Value	Meaning
Fuse Low	`0xFF`	external crystal, full speed, no clock divider
Fuse High	`0xDE`	SPI on, boot section 512 bytes, start at the boot vector
Extended Fuse	`0xFD`	brown-out at 2.7 V
Lock Bits	`0xCF`	boot section protected

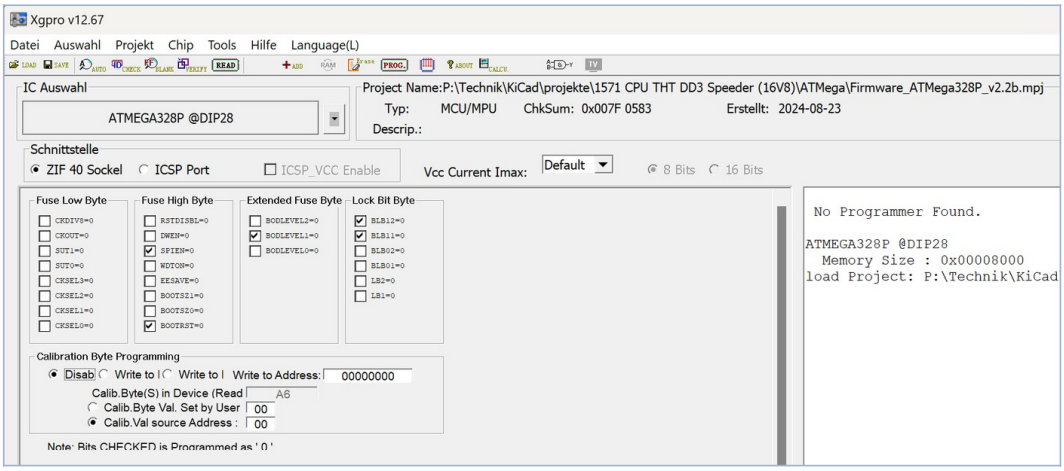


Figure 10: XGecu T48 (Xgpro): fuses of the ATmega328P

With the XGecu T48 and the project file `Firmware_ATMega328P_v2.2b.mpj`, **untick EEPROM** when programming. The EEPROM only holds the last selected bank; programming it as well can cause an error. The firmware clears invalid EEPROM contents itself at the first start.

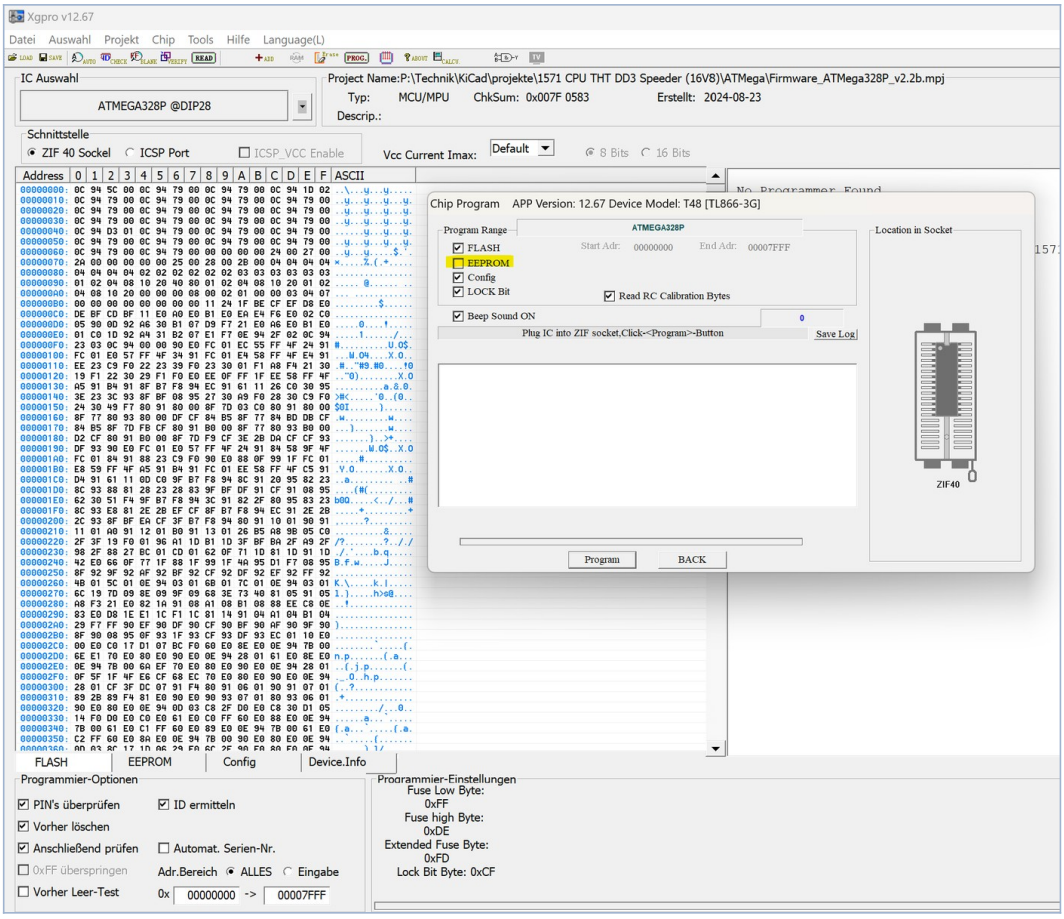


Figure 11: XGecu T48: program FLASH, Config and LOCK Bit, but not EEPROM

With an Arduino as ISP or another AVR programmer, the same works with avrdude, e.g.:

```
avrdude -c usbasp -p m328p -U flash:w:CBM_drive_romswitch.328p_20MHz.hex:i \
        -U lfuse:w:0xFF:m -U hfuse:w:0xDE:m -U efuse:w:0xFD:m
```

The source of the firmware (`CBM_drive_romswitch.ino`) is in the same folder; it can be compiled with the Arduino environment.

7.6 Preparing the EPROMs

The flash chip **29F040** (PLCC32) is recommended for both EPROMs; alternatively a 27C040 or the pin-compatible flash SST39SF040 can be used. Both hold 8 banks of 64 KB. The image is assembled from the individual ROM files and then programmed.

> **MS-ROM-Builder-C128DCR:** It is more convenient with the browser tool `MS-ROM-Builder-C128DCR.html` in the folder `rom-builder` of the 1571 Multi-Speeder project. Download the file and open it in a current browser; it runs without installation and offline on any operating system, and the ROM files never leave the computer. The tool shows both EPROMs (drive U3 and kernal switcher U32) with their eight banks and the slots for 4, 8, 16, 32 and 64 KB, with the templates preset. The ROM files are dropped in and assigned by their checksum or placed directly onto a slot; finished images can be opened and single banks replaced. The tool checks the layout, shows solder jumper J2 for the selected EPROM type and delivers the finished 512 KB file for 29F040, SST39SF040 or 27C040.

> **Solder jumper J2 – set it to match the EPROM type:** The 29F040 and the 27C040 have address line A18 on different pins. The three-pad solder jumper J2 on the Multi-Speeder board (and likewise on the kernal switcher) connects A18 to the right pin. Without the matching jumper the EPROM does not get A18, and the banks are not addressed correctly.

EPROM type (U3)	Pin 1	Pin 31	Solder jumper J2
29F040 (recommended), SST39SF040	A18	/WE	connect **pad 1–2**
27C040 (alternative)	Vpp	A18	connect **pad 2–3**

Pad 2 in the middle carries A18, pad 1 goes to pin 1, pad 3 to pin 31 of the EPROM (see the schematic in chapter 2.2). Only ever connect one side. The **kernal switcher** has the same solder jumper J2 with the same pinout for its EPROM U2; it is set the same way there.

7.6.1 EPROM of the Multi-Speeder board (U3)

The easiest way is to program the ready-made template `C1571_C128DCR_29F040.BIN` (chapter 1.3): banks 1 and 2 and 5 to 8 hold the Commodore DOS 318047-01, bank 3 the original DolphinDOS 3 and bank 4 DolphinDos26. To use other DOS versions, replace single banks or assemble the image yourself:

- Bank 4 (from $30000): `C1571_DOLPHINDOS26.bin`, the complete 64 KB bank
- 32 KB ROMs (e.g. the Commodore DOS 318047-01 for bank 1) into the upper half of their bank (bank start + $8000)
- fill unused banks with a copy of a working bank so that switching there by mistake does not lead nowhere

Example on Linux: create an empty image filled with $FF, fill bank 4 with DolphinDos26 (`seek` = bank − 1 with `bs=64k`) and put the Commodore DOS 318047-01 into the upper half of bank 1:

```
head -c 524288 /dev/zero | tr '\000' '\377' > u3.bin
dd if=C1571_DOLPHINDOS26.bin of=u3.bin bs=64k seek=3 conv=notrunc
dd if=318047-01.bin of=u3.bin bs=32k seek=1 conv=notrunc
```

7.6.2 EPROM of the kernal switcher (U32)

The easiest way is to program the kernal switcher template matching the drive template (chapter 1.3): `C128DCR_KERNAL_29F040-DE.BIN` with the German keyboard for a German C128DCR, otherwise `C128DCR_KERNAL_29F040-US.BIN`. Both use only the upper half of each bank; their banks match `C1571_C128DCR_29F040.BIN`:

Bank	Drive (U3)	Kernal switcher DE	Kernal switcher US
1, 2, 5–8	Commodore DOS 318047-01	C64 BASIC 901226-01, C64 kernal 901227-03, C128 kernal 315078-03 (German)	C64 BASIC 901226-01, C64 kernal 901227-03, C128 kernal 318020-05
3	DolphinDOS 3 (original)	C64 BASIC 901226-01, DolphinDOS 3.0 C64 kernal, Dolphin C128 kernal (original)	as DE
4	C1571DolphinDos26	`C64IIDolphinDos26.bin`, `C128DolphinDos26-DE.BIN`	`C64IIDolphinDos26.bin`, `C128DolphinDos26-US.BIN`

To use other kernals, assemble the image yourself:

In each bank, C64 mode is at + $8000 and the C128 kernal at + $C000, so for bank 4 with DolphinDos26 at $38000 and $3C000:

```
head -c 524288 /dev/zero | tr '\000' '\377' > u32.bin
dd if=C64IIDolphinDos26.bin of=u32.bin bs=16k seek=14 conv=notrunc
dd if=C128DolphinDos26-DE.BIN  of=u32.bin bs=16k seek=15 conv=notrunc
```

For bank n: C64 mode at `seek` = 4 × (n − 1) + 2, C128 kernal at `seek` = 4 × (n − 1) + 3 (each with `bs=16k`). The banks with Commodore DOS get the Commodore kernals (C64 BASIC 901226-01 and kernal 901227-03 or 251913, C128 kernal 318020-05, for the German keyboard 315078-03).

> **JiffyDOS:** Banks 5–8 are free, e.g. for further DOS versions. JiffyDOS is commercial software and not part of the project; anyone who wants to use it needs their own licence from RETRO Innovations.

7.7 Installation in the C128DCR

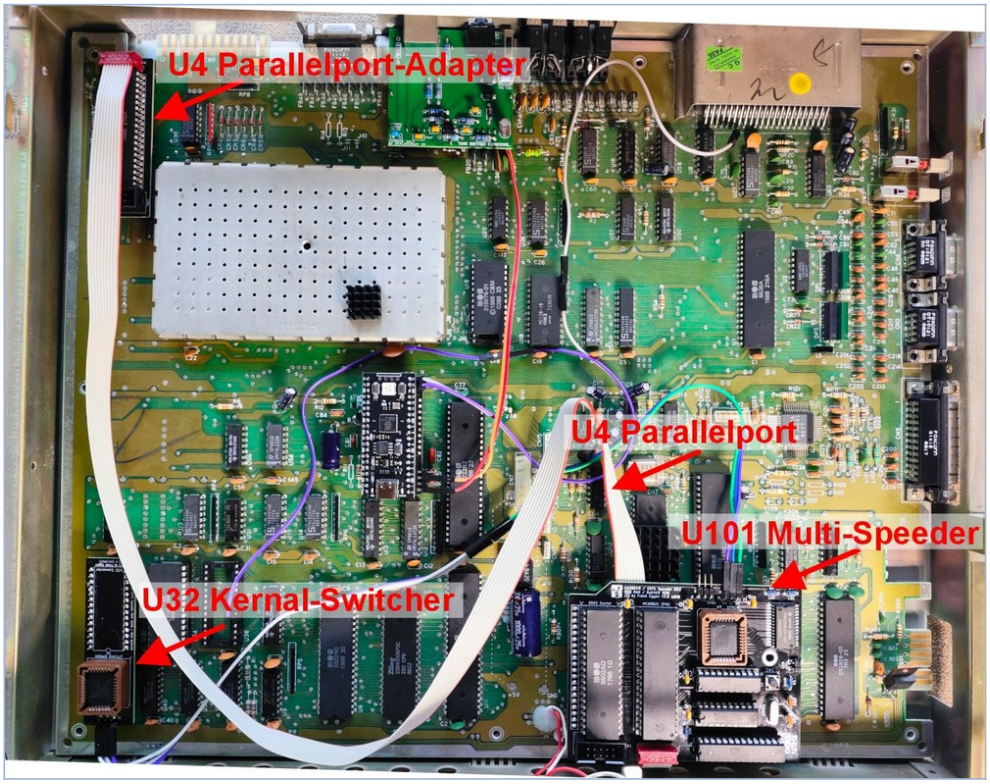


Figure 12: Installation in the C128DCR: U101 Multi-Speeder, U4 parallel port adapter, U32 kernal switcher (photo with version 2.0)

1. Disconnect the C128DCR from the mains, open it and expose the mainboard.
2. **1571 CPU U101:** The 6502 is usually soldered in. Desolder it and fit a 40-pin socket. The CPU later goes into socket U1 of the Multi-Speeder board.
3. **Remove the DOS ROM U102** (without replacement, see chapter 2).
4. **CIA 2 U4:** if soldered in, desolder and socket it. Fit the parallel port adapter and put the CIA into the adapter's socket.
5. **Kernal ROM U32:** take it out and fit the kernal switcher with the programmed EPROM.

6. Put EPROM U3, PIA, 74AHCT273, ATmega and GAL into the Multi-Speeder board, put the 6502 into U1 and press the board into socket U101. Mind the position of pin 1.
7. Route the 10-way Micro-MaTch cable from the parallel port adapter to CN1 or CN3 and connect the line from J5 (A16–A18) to J3 of the kernal switcher.

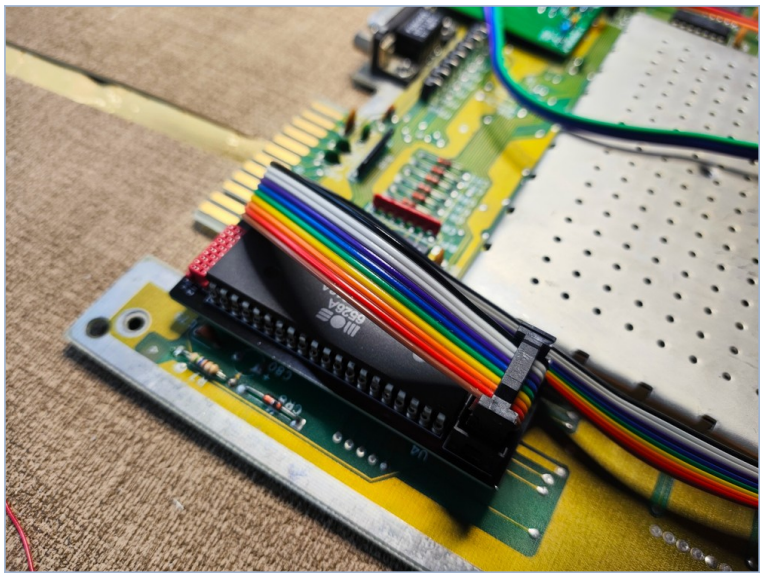


Figure 13: Parallel port adapter in socket U4 with cable (photo with version 2.0)

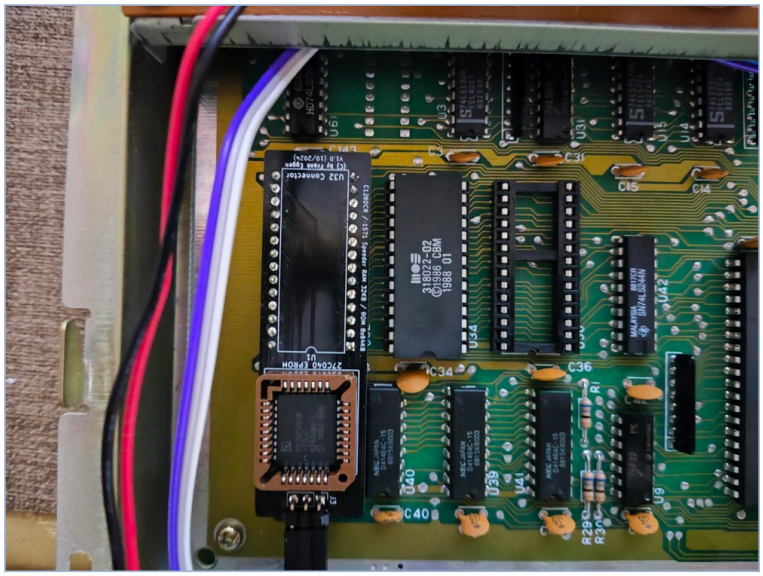


Figure 14: Kernal switcher in socket U32 (photo with version 2.0)

7.8 First start-up

1. Switch on. The LED on the Multi-Speeder board blinks the active bank. With bank 1 the C128 starts as usual.
2. In C128 mode enter `OPEN 1,8,15,"I:4@RNROM":CLOSE 1`; the LED blinks four times. Then restart the C128 with the reset button so that the kernal DolphinDos26 is active.
3. The start-up screen shows the DolphinDos26 lines. `@` reports `73,DOLPHINDOS26 1571,00,00`, `@X` the X status with `P+` for parallel transfer.
4. Load a program: the message `LOADING DD:` confirms parallel transfer.

5. Change to C64 mode with `GO 64`, load `MS-MEMTEST` from the MS-Tools disk and test all 64 banks.

8 Bill of materials (BOM)

The bill of materials follows the KiCad data of version 3.3. The DigiKey and Mouser columns lead to a search for the given type at that distributor; they serve as a reference. Check availability, package and prices before ordering – equivalent parts from other manufacturers are just as suitable.

8.1 Multi-Speeder board

Ref.	Qty	Part	Type / manufacturer no.	DigiKey	Mouser
U3	1	Flash EPROM 512 KB, PLCC32	29F040 (alternatively 27C040 or SST39SF040-70-4C-NHE)	DigiKey	Mouser
U4	1	PIA, DIP-40	W65C21N6TPG-14	DigiKey	Mouser
U5	1	Octal D flip-flop, DIP-20	SN74AHCT273N	DigiKey	Mouser
U6	1	Microcontroller, DIP-28	ATMEGA328P-PU	DigiKey	Mouser
U7	1	GAL/SPLD, DIP-20	ATF16V8B-15PU	DigiKey	Mouser
U8	1	SRAM 512 K × 8, SOP32	AS6C4008-55SIN	DigiKey	Mouser
Y1	1	Crystal 22.1184 MHz, HC-49	22.1184 MHz HC-49	DigiKey	Mouser
C1, C2, C4, C6–C8	6	Ceramic capacitor 100 nF, 2.54 mm pitch	100 nF 50 V X7R	DigiKey	Mouser
C3, C5	2	Ceramic capacitor 22 pF, 2.54 mm pitch	22 pF 50 V C0G	DigiKey	Mouser
R2	1	Resistor 10 kΩ, 1/4 W	MFR-25FBF52-10K	DigiKey	Mouser
R3	1	Resistor 560 Ω, 1/4 W	MFR-25FBF52-560R	DigiKey	Mouser
RN1	1	Resistor network 6 × 10 kΩ, SIP-7, common pin	4607X-101-103LF	DigiKey	Mouser
D1	1	LED 3 mm	LED 3 mm red	DigiKey	Mouser
SW1	1	Push button 6 × 6 mm, through-hole	1825910-6	DigiKey	Mouser
CN1, CN3	2	Micro-MaTch header 10-way for PCB	Micro-MaTch 10-way	DigiKey	Mouser
J3, J5, CN2	3	Pin header 1 × 3, 2.54 mm	pin header 1×3	DigiKey	Mouser
J6	1	Pin header 2 × 3, 2.54 mm	pin header 2×3	DigiKey	Mouser
U3	1	Socket PLCC32, through-hole	PLCC-32 socket THT	DigiKey	Mouser
U1, U4	2	IC socket DIP-40 (precision)	DIP-40 socket	DigiKey	Mouser
U5, U7	2	IC socket DIP-20	DIP-20 socket	DigiKey	Mouser
U6	1	IC socket DIP-28 narrow (7.62 mm)	DIP-28 socket 0.3"	DigiKey	Mouser
U2	1	DIP plug 40-pin (15.24 mm) for the CPU socket U101	334-40-140-00-020000 (Mill-Max)	DigiKey	Mouser

8.2 Parallel port adapter

Ref.	Qty	Part	Type / manufacturer no.	DigiKey	Mouser
CN2	1	Micro-MaTch header 10-way for PCB	Micro-MaTch 10-way	DigiKey	Mouser
U4	1	DIP plug 40-pin (15.24 mm) for socket U4	334-40-140-00-020000 (Mill-Max)	DigiKey	Mouser
U4	1	IC socket DIP-40 (precision) for the CIA	DIP-40 socket	DigiKey	Mouser

8.3 Kernal switcher

Ref.	Qty	Part	Type / manufacturer no.	DigiKey	Mouser
U2	1	Flash EPROM 512 KB, PLCC32	29F040 (alternatively 27C040 or SST39SF040-70-4C-NHE)	DigiKey	Mouser
U2	1	Socket PLCC32, through-hole	PLCC-32 socket THT	DigiKey	Mouser
U1	2	Pin header 1 × 14 for socket U32 (machined)	machined pin header 1×14	DigiKey	Mouser
J3	1	Pin header 1 × 4, 2.54 mm	pin header 1×4	DigiKey	Mouser

8.4 Cables and installation

Ref.	Qty	Part	Type / manufacturer no.	DigiKey	Mouser
–	1	Ribbon cable 10-way, 1.27 mm pitch, approx. 30 cm	ribbon cable 10-way	DigiKey	Mouser
–	2	Micro-MaTch IDC socket 10-way	Micro-MaTch 10-way IDC	DigiKey	Mouser
–	1	Connection J5–J3: 3 wires with female connectors (Dupont)	jumper wire female–female	DigiKey	Mouser
U101, U4	1 each	IC socket DIP-40 for the mainboard (if CPU or CIA are soldered in)	DIP-40 socket	DigiKey	Mouser

9 Appendix

9.1 RAM banks in your own programs

The 512 KB SRAM appears in the drive at $6000–$7FFF; port B of the PIA (PB0–PB5) selects one of the 64 banks. The RAM address lines A13–A18 have pull-up resistors. DolphinDos26 therefore leaves data register B at 0 and writes the **logical bank** n into the **data direction register B**: an output pulls the line to 0, an input lets the pull-up set it to 1. Logical bank n is thus physical bank 63 − n; after power-on (direction register 0), logical bank 0 is active, the working RAM of DolphinDOS. In the example, bit 2 of control register B ($5003) is cleared first, then logical bank 5 is written to $5002; writing 0 returns to the working RAM:

```
        LDA #$00
        STA $5003       ; CRB bit 2 = 0: $5002 = DDRB
        LDA #$05
        STA $5002       ; logical bank 5 at $6000-$7FFF
        ...
        LDA #$00
        STA $5002       ; back to logical bank 0
```

DolphinDos26 uses the banks itself (working RAM in bank 0, CDR pages and RAM disk in banks 1–50). Your own programs should therefore use the MS-Addon-API; the bank layout is in the DolphinDos26 manual.

9.2 Troubleshooting

Problem	Cause and remedy
Drive does not respond after installation	U102 not removed; board the wrong way round or not fully in the socket; GAL not programmed
`@I:4@RNROM` does not switch	repeat the command (2 MHz); use the 22.1184 MHz crystal and a 74AHCT273; check the ATmega fuses
Drive hangs after a change	briefly connect J3 pin 3 to pin 2 (GND): back to bank 1; then check the content of the bank
`LOADING` instead of `LOADING DD:`	check the cable between adapter and CN1/CN3 (pin 1), CIA correctly in the adapter, does `@X` show `P+`?
MS-MEMTEST reports errors	check the solder joints of SRAM U8 and of the resistor network RN1
Program with fast loader does not load	`@X-` (1 MHz) or `@I:1@RNROM` (Commodore DOS), see chapter 4.2

9.3 Origin and acknowledgements

Project	Developers	Contribution
DolphinDOS 3.0	Günther Jilg (software) and Jan Bubela (hardware), Dolphin Software	Basis of DolphinDos26 and the Dolphin kernals
Switchless Multi-ROM	RetroNynjah	Model for the bank switching with the microcontroller
Review V2.1	Jim Drew and Joeri van Haren	Help and review of the hardware

Project	Developers	Contribution
DolphinDOS 3 data	silverdr	Technical data on DolphinDOS 3
1571 TrackCache	ytmytm	Track cache patch for the Commodore DOS of the 1571
Idea	Stefan Kauf	Support with the idea of the Multi-Speeder
JiffyDOS	Mark Fellows; Creative Micro Designs, today Jim Brain, RETRO Innovations	JiffyDOS protocol in the DolphinDos26 kernals

Special thanks go to Jim Brain of RETRO Innovations for his support with the JiffyDOS implementation of the DolphinDos26 kernals.

> **Please check: JiffyDOS licence.** The kernals `C128DolphinDos26-US.BIN`, `C128DolphinDos26-DE.BIN` and `C64IIDolphinDos26.bin` contain JiffyDOS transfer routines that are instruction-identical to a JiffyDOS kernal (C128: JiffyDOS C128 kernal, C64: JiffyDOS sd2iec kernal); the JiffyDOS firmware for the drive is not included. Jim Brain of RETRO Innovations, which holds the JiffyDOS licence rights, confirmed that publishing the DolphinDos26 kernals on GitHub is fine. As he describes it, code that implements the same transfer behaviour without being an exact copy of the JiffyDOS routines needs no licence. If the transfer code is the same, as it is here, he believes the end user would need one. Every user is therefore asked to check for themselves whether they should purchase an official JiffyDOS licence because of their use. Further information: www.jiffydos.com

Multi-Speeder, DolphinDos26 and this manual: **Frank Eggen**.

Topic	Address
Contact	frankeggen@emden.net
Projects on GitHub	https://github.com/FraEgg
DolphinDOS26: firmware, kernals, MS-Tools, manuals	github.com/FraEgg/DolphinDOS26
Switchless Multi-Speeder for the 1541 (hardware)	github.com/FraEgg/commodore-1541-switchless-floppydrive-8x-multi-floppy-speeder
Switchless Multi-Speeder for the C128DCR with 1571 (this project)	github.com/FraEgg/Switchless-Multi-Speeder-Commodore-C128DCR-1571
Order the boards at PCBWay	Switchless 8x Multi-Floppy-Speeder for the C128DCR
Switchless Multi-Speeder for the C128DCR with 1571 (previous release)	github.com/FraEgg/commodore-c128dcr-1571-switchless-floppydrive-8x-multi-floppy-speeder

9.4 Disclaimer

The 1571 Multi-Speeder and DolphinDos26 are a **private hobby project** and are provided without any warranty. Building, installation and use are **at your own risk**; no liability is accepted for damage to computer, drive, disks, data or any other consequences.

9.5 Supporting the project with a donation

Anyone who would like to support the project can send the developer a voluntary donation via PayPal:

> **PayPal:** Donation to Frank Eggen

9.6 Copyright and terms of use

Part	Rights holder
Hardware of the Multi-Speeder, extensions of DolphinDos26, kernals DolphinDos26, MS-Tools, this manual	© 2024–2026 Frank Eggen
DolphinDOS 3.0	© Günther Jilg and Jan Bubela, Dolphin Software
JiffyDOS	© Mark Fellows / Creative Micro Designs; licensing today through Jim Brain, RETRO Innovations
CBM DOS 1571, C128 and C64 kernal, BASIC	© Commodore

Hardware, software and this manual may be used, rebuilt and passed on for **private, non-commercial use**. **No sale and no commercial use without consulting the developer** (in particular selling boards, kits, ROM files or programmed EPROMs). Please send enquiries to frankeggen@emden.net.